Singular Value Decomposition

Truncated SVD by power iteration

The problem

- Large matrix A (users × items, documents × words)
- Want its best rank-k approximation
- Without a full, expensive decomposition

SVD

$$A = U\Sigma V^\top = \sum_i \sigma_i \, u_i v_i^\top$$

- $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$
- Top-k terms = best rank-k approximation (Eckart–Young)

Power iteration

- Start with a random unit vector v
- Repeat: $v \leftarrow A^T A\, v$, normalise
- v converges to the top right singular vector
- $\sigma = \|Av\|$, $u = Av / \sigma$

Deflation

- Subtract the found piece: $R \leftarrow R - \sigma\, u\, v^{T}$
- Repeat power iteration on R for the next one
- k rounds → k components

truncated_svd

```
def truncated_svd(A, k, iters=500, seed=0):
    A = np.asarray(A, float)
    rng = np.random.default_rng(seed)
    U, S, V = [], [], []
    R = A.copy()
    for _ in range(k):
        v = rng.normal(size=A.shape[1])
        v /= np.linalg.norm(v)
        for _ in range(iters):
            v = R.T @ (R @ v)
            nv = np.linalg.norm(v)
            if nv == 0:
                break
            v /= nv
        s = np.linalg.norm(R @ v)
        u = R @ v / s if s > 0 else np.zeros(A.shape[0])
        U.append(u); S.append(s); V.append(v)
        R = R - s * np.outer(u, v)
    return np.array(U).T, np.array(S), np.array(V)
```

Complexity & trade-offs

- O(k · iters · n · d), cheap for small k
- Slow if σ_k ≈ σ_{k+1} (small spectral gap)
- Errors accumulate through deflation
- Production: randomized SVD, Lanczos

Recap

- A = sum of $\sigma\ u\ v^T$ pieces
- Power iteration finds the top piece
- Deflate and repeat
- Basis for PCA, LSA, recommenders